clautolisp Debugger — Implementation Plan

Implementation roadmap for clautolisp-debugger-spec v0.1

June 1, 2026

Contents

1 0. Purpose and scope **1**
- 1.1 0.1 Primary target ordering (decision) . . . 1

2 1. Current-codebase findings (what we build on, what is missing) **2**
- 2.1 1.1 Already present (reuse) . . . 2
- 2.2 1.2 The load-bearing gap: source positions are stripped before evaluation . . . 3
- 2.3 1.3 Remote transport gap (affects Phase 9 only) . . . 3

3 2. New ASDF systems and packages **3**

4 3. Phased plan **4**
- 4.1 3a. The interpreter two-bodies mechanism (§5 pulled forward; supersedes §11.3) . . . 4
- 4.2 Phase 0 — Source map and protocol skeleton (§3, §4, §7, §8) 6
- 4.3 Phase 1 — Interpreter instrumentation, poll points, breakpoints (§5.3, §6, §11, §12) . . . 6
- 4.4 Phase 2 — Snapshot, environment, stepping (§6, §9) . . . 7
- 4.5 Phase 3 — AutoLISP error integration (§10) . . . 7
- 4.6 Phase 4 — Inspector (Part IV) . . . 8
- 4.7 Phase 5 — UI protocol + dumb-terminal UI (§17, §18, §21–§24) 8
- 4.8 Phase 6 — ncurses UI (§19) . . . 9
- 4.9 Phase 7 — Emacs UI (aldb) (§20) . . . 9
- 4.10 Phase 8 — Compiler two-bodies (§5, §13) [deferred / v2] . . . 9
- 4.11 Phase 9 — Remote CAD backend (addendum R-II..R-VIII) [later] . . . 9

5 4. Issue breakdown (issues/open/) 10

6 5. Risks and decisions 11

7 6. References 11

1 0. Purpose and scope

This is the **implementation plan** for the clautolisp debugger, inspector, and the three pluggable user interfaces. It turns two normative documents into ordered, shippable work:

- `clautolisp-debugger-spec.org` — the parent spec (in-image debugger against the clautolisp interpreter). Section refs like §9 are to it.
- `clautolisp-debugger-remote-cad-spec.org` — the addendum (remote native-AutoLISP backend driven from alfe). Section refs like R-III are to it.

Tracking issue: `issues/open/clautolisp-debugger.issue`.

1.1 0.1 Primary target ordering (decision)

The **in-image debugger** (parent spec) is v1. It targets the clautolisp **interpreter**, which already exists, and is what the three UIs and the inspector are specified against. The **remote-CAD backend** (addendum) is a later phase: it is an explicit addendum that **reuses** the debugger library, inspector, UIs, reader, and instrumenter unchanged, and it additionally requires transport work that the current alfe file-IPC does not satisfy (see §1.3). Building in-image first de-risks everything the remote phase reuses.

The **two-bodies discipline** (parent §5, Raskin) is pulled forward into the **interpreter** rather than deferred to the compiler. Instead of the parent's §11.3 "single AST gated by a `*debug-mode*` flag in the evaluator" variant (rejected: it pays a per-sub-form flag check even for code nobody is debugging), each instrumentable `autolisp-usubr` carries **two bodies**: the plain `body` (always) and an `instrumented-body` (only when instrumented). The ordinary body has **no** poll-point nodes, so un-debugged execution costs zero — not even a flag check. See §3a for the mechanism. The *compiler* two-bodies variant (parent §5.1, emitting CL) stays deferred to Phase 8; this interpreter scheme is its AST-level analog, which makes that later migration conceptually smaller.

2 1. Current-codebase findings (what we build on, what is missing)

2.1 1.1 Already present (reuse)

- **Reader source tracking (parent §3, partial).** The reader attaches a `source-span` (`source-name`, start/end line/column) to **every** parsed object — `symbol-object`, `string-object`, `integer-object`, `real-object`, `cons-object`, `quote-object`, `comment-object`, `dot-object`. See `autolisp-reader/source/model.lisp:3` (`source-span`) and the per-object `-span` slots. Package: `clautolisp.autolisp-reader` (impl in `.internal`). `combine-spans` merges spans (`model.lisp:107`).
- **Tree-walking evaluator.** `autolisp-eval` (`autolisp-runtime/source/api.lisp:1793`) dispatches special operators via `*special-operator-dispatch*` (`api.lisp:1755`: QUOTE/SETQ/IF/WHILE/FOREACH/DEFUN/DEFUN-Q/…) and calls functions via `call-autolisp-function-in-context` (`api.lisp:1379`).
- **Dynamic binding model (maps to parent §9).** `dynamic-frame` is a stack of `(bindings . parent)` hash tables (`model.lisp:66`); per-symbol `binding-cell` holds value/bound-p (`model.lisp:25`); `evaluation-context` carries session/document/namespace/dynamic-frame (`model.lisp:117`). User functions are `autolisp-usubr` structs (name/lambda-list/body/environment, `model.lisp:153`).
- **Call-stack capture.** `*autolisp-call-stack*` is a list of `(kind . form)` frames (`api.lisp:76`), captured into every error.
- **Structured errors.** `autolisp-runtime-error` condition (code/message/details/call-stack, `api.lisp:87`); `signal-autolisp-runtime-error` (`api.lisp:125`); per-session `errno` (`api.lisp:456`).
- **Threads.** `bordeaux-threads` is already a dependency (front-end); reuse for the two-thread debugger model and the per-thread queues.

2.2 1.2 The load-bearing gap: source positions are stripped before evaluation

`reader-object->runtime-value` (`api.lisp:1097`) lowers reader objects to **bare runtime values**: a `cons-object` (which carries a span) becomes a plain CL list, an `integer-object` becomes a bare integer, etc. The evaluator runs on these stripped values, so **no source position survives into**

evaluation today. The parent spec §3 requires the exact opposite: every form carried by the evaluator must resolve to a source position via `clautolisp.source:position-of`.

This is the first thing to build (Phase 0). Because bare atoms (fixnums, interned symbols) cannot be uniquely `EQ`-keyed, the chosen mechanism is a **per-function form-id table** assigned at lowering time (parent §7, §11): the lowering pass walks the `cons-object` tree, assigns dense form-ids, records `form-id → source-span`, and keys runtime **compound** forms by `EQ` (fresh conses are unique) plus the form-id table for atoms. This is also exactly the metadata the instrumenter and stepper need.

2.3 1.3 Remote transport gap (affects Phase 9 only)

alfe CAD today is **file-IPC**: atomic file writes + status polling (`autolisp-front-end/source/file-pr` files status/stdin/ stdout/stderr/control/heartbeat). It is batch/poll oriented and **cannot be read mid-evaluation**, so it fails the addendum's `REQ-T1` (synchronous mid-evaluation readability). Consequence: remote **live** breakpoints and stepping need a new control channel (socket/pipe); otherwise only **post-mortem mode** (break-on-error + reconstructed backtrace) is conformant. Phase 9 budgets for this explicitly.

3 2. New ASDF systems and packages

All new systems live under the `clautolisp/` aggregate and follow the existing `clautolisp/<subsystem>` + `.../tests` pattern in `clautolisp/clautolisp.asd`. Naming follows the project convention (`clautolisp.<area>` packages; `clal-` prefix for user-visible AutoLISP extension functions/commands).

System	Package(s)	Spec part
`clautolisp/autolisp-source-map`	`clautolisp.source`	§3, §7
`clautolisp/autolisp-debug`	`clautolisp.debug`	§4, §8, §1
`clautolisp/autolisp-debug-engine`	`clautolisp.debug` (snapshot/step)	§6, §9, §1
`clautolisp/autolisp-inspect`	`clautolisp.inspect`	Part IV
`clautolisp/autolisp-debug-ui`	`clautolisp.debug.ui`	§17, §21–
`clautolisp/autolisp-debug-ui-dumb`	`clautolisp.ui.dumb`	§18
`clautolisp/autolisp-debug-ui-tui`	`clautolisp.ui.tui`	§19 (cl-ch
`clautolisp/autolisp-debug-ui-emacs`	`clautolisp.debug.remote.rpc` + elisp `aldb`	§20
`clautolisp.debug.remote` (in alfe)	`clautolisp.debug.remote`, `aldbg:` (.lsp)	addendur

New third-party deps: `bordeaux-threads` (debug engine), `cl-charms` (ncurses UI; PDCurses via CFFI on Windows as the alternate backend behind the `clautolisp.ui.tui` abstraction).

4 3. Phased plan

Each phase lists deliverables, the spec sections it satisfies, and an acceptance check. Phases are serial except where noted; UIs (Phase 5–7) can proceed in parallel once Phase 5's protocol exists. Every code-bearing phase adds FiveAM tests in the matching `.../tests` system and bumps the version. **Version policy:** the debugger is a significant feature addition to clautolisp *and* alfe, so the series starts at **1.1.0** (a minor bump from the current 1.0.x) and climbs from there. Bump `tools/clautolisp/source/version.lisp` and `autolisp-front-end/tools/alfe/source/version.lisp`.

Both version files move into the 1.1 series on this branch, even though the bulk of the per-change DEVELOP bumps so far are clautolisp's. alfe is pinned at **1.1.0** as its 1.1-series starting point; its DEVELOP counter only climbs once alfe *source* actually changes on this branch (Phase 9 — the remote-CAD backend — and any alfe-side wiring of the Emacs RPC). Until then alfe stays at 1.1.0, which is also already ahead of master's 1.0.x, so the merge-back convention (new = 0.0.1 + max(ours, theirs)) keeps the debugger branch ahead regardless of how master's alfe climbs within 1.0.x.

4.1 3a. The interpreter two-bodies mechanism (§5 pulled forward; supersedes §11.3)

:PROPERTIES: :CUSTOM_ID: two-bodies-interp :END:

This refines how Phases 0–1 instrument interpreted code, replacing the parent §11.3 evaluator-flag variant.

- **Two bodies per usubr.** `autolisp-usubr` gains an `instrumented-body` slot (and a `debug-metadata` slot). `body` (the plain AST) is always present; `instrumented-body` exists only for functions that have been instrumented. Builtin =autolisp-subr=s are opaque CL and are never instrumented — they are stepped *over*, consistent with §6 and the opaque-code limits.
- **The instrumented body is woven AST, not a flag.** An instrumentation pass produces `instrumented-body` as a **copy** of `body` with poll-point nodes spliced in around every sub-form — `(%poll`

`k :before) … (%poll k :after)` — plus function enter/exit nodes (parent §5.3 shape), where `%poll` is a debug special operator registered in `*special-operator-dispatch*`. The ordinary `body` has no `%poll` nodes, so un-debugged evaluation costs **zero** (no per-form flag check). The cheap fast path (debug-flag → Bloom → exact lookup, parent §11.1) lives **inside** `%poll`'s routine. This pass is folded into the Phase-0 form-id traversal — no separate walk.

- **Body selection at the apply boundary (per function; see DN-2).** A dynamic `*debugging*` indicator means "a debug session is active on this thread." At each function *application*, `call-autolisp-function-in-context` picks the callee's `instrumented-body` if `*debugging*` is set and the callee has one, else the plain `body`. *Originally this also propagated debugging-ness along the call chain (clearing `*debugging*` inside un-instrumented bodies); design note DN-2 removed that.* Selection is now **per function**: `*debugging*` is set once by the session entry and not rebound per call, so an instrumented function is debugged wherever it is called from — including through an un-instrumented library function, a HOF, or a builtin — while an un-instrumented function always runs plain. This is the interpreter twin of the remote public-name swap (R-II.2) and is consulted only at call sites; it stays correct under REPL redefinition (callees resolve late, by symbol).

- **Why not §11.3.** The flag variant pays a branch at every sub-form of every function evaluated in a debugged dynamic extent, even functions nobody is debugging, and has no real "ordinary vs debug body" distinction. The two-bodies scheme runs poll points only inside instrumented bodies, gives zero overhead for un-instrumented functions (no poll nodes at all), and is a smaller eventual jump to the Phase-8 compiler (same discipline, AST→CL).

4.2 Phase 0 — Source map and protocol skeleton (§3, §4, §7, §8)

:PROPERTIES: :CUSTOM_ID: phase0 :END:

- **0.1 `clautolisp.source`.** A `source-position` object = `(start . end)` of `(file line column)`, `EQUAL`-comparable and printable; `position-of object` and `lines-of file` (parent §3.4).

- **0.2 Form-id lowering.** Extend the reader→runtime lowering so that, per top-level/defun form, it assigns dense contiguous form-ids, builds

`form-id → source-position` and `form-id → kind` and `parent-form-map` vectors, and an `EQ` side-table from runtime compound forms to form-ids. This **closes the §1.2 gap.** Keep the non-debug lowering path unchanged for production speed.

- **0.3** `function-debug-metadata` (parent §7) stored in a new `debug-metadata` slot on `autolisp-usubr` (alongside the new `instrumented-body` slot, §3a): source-position, form-id→position, form-id→kind, poll-point-count, bound-names (formals + `/`-locals in binding order), parent-form-map, source-text.

- **0.4** `clautolisp.debug` **skeleton:** `*protocol-version*`, `thread-debug-info` (debug-flag, breakpoint-table, 1024-bit summary, inbound/outbound queues, current-pp, status; parent §8.1), `*thread-debug-info-table*`.

Acceptance: `(position-of form)` returns the right `(file line col)` for a form pulled mid-evaluation; metadata round-trips for a sample `defun`.

4.3 Phase 1 — Interpreter instrumentation, poll points, breakpoints (§5.3, §6, §11, §12)

:PROPERTIES: :CUSTOM_ID: phase1 :END:

- **1.1 Instrumentation pass + two-body dispatch** (§3a). Build `instrumented-body` by weaving `%poll` nodes (before/after each subform, plus function enter/exit) into a copy of `body`; register the `%poll` debug special operator. Make `call-autolisp-function-in-context` select `instrumented-body` vs `body` at the apply boundary via `*debugging*` (§3a). This replaces the parent §11.3 evaluator-flag variant.

- **1.2** `poll-point` with the parent §11.1 hot path: debug-flag fast path → 1024-bit Bloom test (§11.2) → exact breakpoint lookup → `handle-breakpoint`. Allocation-free hot path.

- **1.3 Breakpoint API:** `add-breakpoint=/=remove-breakpoint=/ =list-breakpoints` (parent §12); steady vs volatile; Bloom-bit set on add, lazy rebuild on shrink.

- **1.4 Two-thread pause:** application thread enqueues `:hit` and blocks on its inbound queue; debugger thread drives it; `:continue` resumes (parent §8.2/§8.3).

Acceptance: set a steady breakpoint on a form in a loaded `.lsp`; running it stops at the form; `:continue` resumes; Bloom filter measurably skips non-breakpoint poll points.

4.4 Phase 2 — Snapshot, environment, stepping (§6, §9)

:PROPERTIES: :CUSTOM_ID: phase2 :END:

- **2.1 Snapshot producer** (parent §9.2): function, form-id, source-position, call-stack (from `*autolisp-call-stack*` + `dynamic-frame`), binding-stack, visible-names, globals-touched, catch-stack. Map `dynamic-frame=/=binding-cel`
  onto the spec's `stack-frame=/=binding-entry`.
- **2.2 Frame navigation + writes:** `with-frame-bindings` (parent §9.3), `cmd-set-variable=/binding-entry writes incl. shadowed entries (§9.4, §9.5), =coerce-from-cl`.
- **2.3 Stepping** as volatile breakpoints (parent §6): in/out/over/call/finish using `parent-form-map` + function-entry/exit poll points; `poll-point-at source-position` for "advance to point".

Acceptance: step over/in/out across a recursive `defun`; inspect a shadowed binding; rewrite a binding and observe the change after `:continue`.

4.5 Phase 3 — AutoLISP error integration (§10)

:PROPERTIES: :CUSTOM_ID: phase3 :END:

- **3.1 `*error*` interception** (parent §10.1): on the error path build a snapshot, send `:unhandled-error`, block; resolutions continue-with-error / continue-with-return / abort. Add the runtime hook if absent; integrate with `autolisp-runtime-error` + `errno`.
- **3.2 `vl-catch-all-apply`** (parent §10.2): record catch frames in per-thread `catch-stack`; optional break-on-caught with `:caught-error`.

Acceptance: `(/ 1 0)` breaks into the debugger with the erroring form highlighted; abort restores sysvars; a `vl-catch-all-apply` frame shows in the snapshot.

4.6 Phase 4 — Inspector (Part IV)

:PROPERTIES: :CUSTOM_ID: phase4 :END:

- **4.1 `clautolisp.inspect`:** `inspect`, `inspector-session`, `inspect-page-for` generic + per-type methods/accessors for every AutoLISP type (number/string/symbol/cons/ename/ssget/vla/file/hash; parent §15.1).
- **4.2 Path expressions** (parent §15.2) incl. `:opaque` partial paths.
- **4.3 Workspace `$N`** (parent §16.2) + `session-bind` destinations (:workspace/:global/:frame/:setq parent §16.1).

Acceptance: inspect an ename, descend DXF group 10, copy path `(cdr (assoc 10 (entget EN)))`, bind to `$4`, reference `$4` in eval.

4.7 Phase 5 — UI protocol + dumb-terminal UI (§17, §18, §21–§24)

:PROPERTIES: :CUSTOM_ID: phase5 :END:

- **5.1 `clautolisp.debug.ui` protocol:** the `ui-*` notifications and `cmd-*` commands (parent §17.1/§17.2) as CLOS generics; session lifecycle =start-session=/=with-session=/=attach-thread=/teardown (parent §21–§24).
- **5.2 Dumb-terminal UI** (parent §18): line-oriented sub-REPL, single-letter commands, numbered source listing with `>>` marker, inspector in dumb terminal (§18.2). This is the bring-up UI used to validate every phase above.

Acceptance: a full break→inspect→step→continue session over plain stdin/stdout; usable in CI and over a pipe.

4.8 Phase 6 — ncurses UI (§19)

:PROPERTIES: :CUSTOM_ID: phase6 :END:

- `clautolisp.ui.tui` thin abstraction over cl-charms (Unix) / PDCurses (Windows); four-pane layout (stack/source/interactor/repl); poll-point/ breakpoint/current markers; inspector pane; UI-thread-only curses calls with a notification queue (parent §19.1–§19.3).

Acceptance: keyboard-only navigation complete; 8-colour fallback; mouse optional.

4.9 Phase 7 — Emacs UI (aldb) (§20)

:PROPERTIES: :CUSTOM_ID: phase7 :END:

- Line-oriented S-expression RPC (parent §20.1) + Emacs minor mode `clautolisp-debug-mode` / `aldb` with the SLDB-modelled buffers and keybindings (parent §20.2/§20.3); source overlays at poll points/ breakpoints.

Acceptance: SLDB-parity feel for break/step/inspect from Emacs against an in-image session.

4.10 Phase 8 — Compiler two-bodies (§5, §13) [deferred / v2]

:PROPERTIES: :CUSTOM_ID: phase8 :END:

When the compiler-to-CL lands: emit `foo/ordinary` + `foo/debug` behind an entries vector (parent §5.1), poll points only in the debug body, Fjeld entry-point propagation. The debug protocol and all UIs are unchanged (parent §13).

4.11 Phase 9 — Remote CAD backend (addendum R-II..R-VIII) [later]

:PROPERTIES: :CUSTOM_ID: phase9 :END:

Follows the addendum's own phasing (R-XI), gated on transport:

1. **Transport + framing** (`REQ-T1..T3`, `P1..P3`). The current file-IPC fails `REQ-T1`; add a synchronous mid-evaluation channel (socket/pipe) **or** ship post-mortem mode and advertise it via capability flags (R-VIII §11). Prove a pause nests inside an in-flight `:eval` and one `:eval-in-frame` round-trips.

2. **`aldbg:` AutoLISP runtime** on the backend: `*debug-flag*`, breakpoint table, Bloom bits, `aldbg:pp`, `aldbg:enter/exit`, pause loop (R-III.1–R-III.3).

3. **`clautolisp.debug.remote` + retargeted instrumenter:** `autolisp-compile[-file]` emitting `~ord=/=~dbg` AutoLISP bodies with source-level Fjeld propagation (R-II); all §7 metadata stays in alfe.

4. Breakpoints/stepping/backtrace/innermost eval end-to-end (the "Full" rows of R-IX).

5. Inspector over the wire: proxies + backend accessor evaluation (R-IV).
6. Error integration: `*error*`, `vl-catch-all`, ERRNO, continue-with-return for instrumented forms (R-III.7).
7. "Partial" rows: shadowed-binding read/write, outer-frame eval, conditions (set-time `$N` snapshot), tracing, cooperative interrupt; then hardening (reactors, command-input, namespaces, BricsCAD/AutoCAD shim).

5 4. Issue breakdown (issues/open/)

One issue per phase (and per UI), tracked under the parent `clautolisp-debugger.issue`; deferred items follow the `deferred-<slug>.issue` convention:

- `debugger-source-map.issue` — Phase 0
- `debugger-instrumentation.issue` — Phase 1
- `debugger-snapshot-stepping.issue` — Phase 2
- `debugger-error-integration.issue` — Phase 3
- `debugger-inspector.issue` — Phase 4
- `debugger-ui-protocol-and-dumb.issue` — Phase 5
- `debugger-ui-ncurses.issue` — Phase 6
- `debugger-ui-emacs-aldb.issue` — Phase 7
- `deferred-debugger-compiler-two-bodies.issue` — Phase 8
- `deferred-debugger-remote-cad.issue` — Phase 9

6 5. Risks and decisions

1. **Source positions through lowering (Phase 0)** is the critical path; everything downstream depends on `position-of` working on evaluated forms. Mitigation: form-id table + `EQ` side-table assigned at lowering, validated first.
2. **`*error*` hook** may not exist in the runtime yet; Phase 3 budgets for adding it. Confirm during Phase 3 kickoff.

3. **Remote transport (`REQ-T1`)** is unmet by file-IPC; Phase 9 may ship post-mortem mode first. Open question R-XII.3 to reconcile.

4. **Performance of the interpreter debug path:** acceptable per parent §13 ("debugged code is the slow path by definition"); the Bloom filter keeps non-breakpoint poll points cheap.

7 6. References

See the two specs' bibliographies. Core design lineage: Strandh, *Omnipresent and low-overhead application debugging* (ELS 2020) and the Clordane prototype; Fjeld (multiple entry points) and Raskin (two bodies); SLDB/SLIME for the Emacs UI; Autodesk *AutoLISP Developer's Guide* for `*error*` and `vl-catch-all-apply`.